1. Inputs and coordinate conventions

What the two conversion functions calculate

The calculation takes an event sky position and UTC trigger time, rotates the direction into the Earth-fixed frame used by GWAK, and projects a gravitational-wave polarization basis onto the Hanford and Livingston detector arms.

Inputs and units

Symbol	Meaning	Code input
alpha	Right ascension, degrees	ra_deg
delta	Declination, degrees	dec_deg
UTC	Full trigger date and clock time	trigger_time_utc
DUT1	UT1 minus UTC, seconds	ut1_minus_utc_seconds
Delta AT	TAI minus UTC, seconds	tai_minus_utc_seconds
psi	Polarization angle, radians	polarization_rad

RA is in [0, 360) degrees; declination is in [-90, 90] degrees. All trigonometric calculations use radians. Psi defaults to zero as a reference basis, not an inferred physical polarization.

Earth-fixed axes and angles

+z points north along the rotation axis. +x points toward the equator at Greenwich; +y points toward the equator at 90 degrees east. Phi increases from +x toward +y. Declination is measured up from the equatorial plane; polar angle theta is measured down from +z.

$$\alpha = \alpha_{\text{deg}}\,\pi/180, \quad \delta = \delta_{\text{deg}}\,\pi/180$$

Scope of “from first principles”

The arithmetic implementation uses Gregorian calendar arithmetic, published IAU Earth-rotation/sidereal-time formulas, and detector tensor contractions. It does not call Astropy or ERFA. The IAU coefficients, leap seconds, arm vectors and measured DUT1 are inputs, not quantities derived from fundamental physics here.

Both paths preserve the project convention phi = RA - GMST, with declination unchanged. This is not a full ICRS/J2000-to-ITRS transformation: source precession, nutation and polar motion are not applied.

2. Calendar and time scales

Code: _calendar(), _tai_offset(), and the first calculation block

Let Y, M, D be the Gregorian year, month and day. Let h, m_clock and s be the UTC hour, minute and possibly fractional second. Floor brackets correspond to Python integer division //.

$$a = \lfloor (14 - M)/12 \rfloor, \quad y = Y + 4800 - a, \quad m = M + 12a - 3$$

$$J = D + \lfloor (153m + 2)/5 \rfloor + 365y + \lfloor y/4 \rfloor$$

$$- \lfloor y/100 \rfloor + \lfloor y/400 \rfloor - 32045$$

$$JD_0 = J - 0.5, \qquad s_{\text{day}} = 3600h + 60m_{\text{clock}} + s$$

J is the integer Julian day number at noon. Subtracting 0.5 gives midnight at the start of the UTC date. The code keeps the day and its fraction separate to preserve floating-point precision.

Two clocks are needed

UT1 follows the actual rotation of Earth. TT (Terrestrial Time) is a uniform time scale used for the sidereal-time polynomial. DUT1 must come from Earth-orientation measurements; leap-second history supplies Delta AT.

$$UT1 = UTC + DUT1, \qquad TT = UTC + \Delta AT + 32.184 \text{ s}$$

$$f_{UT1} = \frac{s_{\text{day}} + DUT1}{86400}, \qquad f_{TT} = \frac{s_{\text{day}} + \Delta AT + 32.184}{86400}$$

$$JD_{UT1} = JD_0 + f_{UT1}, \qquad JD_{TT} = JD_0 + f_{TT}$$

If DUT1 is omitted, the independent function explicitly assumes UT1 = UTC. For exact comparison, the verifier supplies the same measured DUT1 used by the Astropy path. TAI-UTC is selected independently from the dated leap-second table.

The bundled table covers dates from 1972-01-01 until its expiry, 2027-06-28; outside that range an explicit verified TAI-UTC is required. Literal leap-second timestamps ending in :60 are rejected. Ordinary times on either side of leap changes are tested.

3. Earth rotation and sidereal time

Code: ut1_days, tt_centuries, era_turns, offset_arcsec, gmst

Measure time from J2000

$$d = JD_{UT1} - 2451545.0, \qquad T = \frac{JD_{TT} - 2451545.0}{36525}$$

d is elapsed UT1 days from the J2000 reference Julian date. T is elapsed TT Julian centuries. A Julian century is 36,525 days.

Earth rotation angle (ERA)

$$ERA = 2\pi\,[0.7790572732640 + 1.00273781191135448\,d] \bmod 2\pi$$

The implementation removes integer turns before summation. Since JD0 is an integer minus 0.5, d modulo one turn is equivalent to 0.5 + f_UT1. Thus the following expression has the same angle, with less precision lost to large numbers:

$$ERA = 2\pi\,[0.7790572732640 + 0.00273781191135448\,d + 0.5 + f_{UT1}] \bmod 2\pi$$

IAU 2006 Greenwich mean sidereal time

The polynomial P(T) is in arcseconds. It accounts for the offset between the Earth rotation angle and mean equinox-based sidereal time; it does not transform the input catalog source coordinates.

$$P(T) = 0.014506 + 4612.156534T + 1.3915817T^2 - 0.00000044T^3 - 0.000029956T^4 - 0.0000000368T^5$$

$$g = GMST = \left(ERA + \frac{\pi}{180 \times 3600}\,P(T)\right) \bmod 2\pi$$

math.fsum sums the polynomial terms. The Astropy path requests mean Greenwich sidereal time with model IAU2006, using the same convention and the same measured UT1-UTC input.

4. Earth-relative source direction

Code: phi, declination, theta, vector, and gwak_sampling

Earth has turned through Greenwich sidereal angle g. Subtract this rotation from the celestial right ascension to get the Earth-relative azimuth. The minus sign is essential: at fixed RA, increasing sidereal time decreases phi.

$$\phi = (\alpha - g) \bmod 2\pi, \quad \delta_E = \delta, \quad \theta = \pi/2 - \delta_E$$

$$\hat{s} = (\cos\delta_E \cos\phi,\ \cos\delta_E \sin\phi,\ \sin\delta_E)$$

The unit vector s points FROM Earth TOWARD the source. The wave propagation direction is -s. This geocentric direction is shared by H1 and L1 in the distant-source approximation; detector response differences come from their arm orientations.

Connection to the training parameters

Returned quantity	GWAK sampler parameter
phi (radians)	fixed_phi_rad
delta_E (radians)	fixed_declination_rad
W (degrees, full width)	sky_window_deg

The default W = 1 degree is the same full-width box used in our earlier studies: +/-0.5 degree in each coordinate. It is not a cone radius, an equal-solid-angle patch or an event localization uncertainty.

$$\phi_{\text{sample}} = (\phi + \varepsilon_\phi) \bmod 2\pi, \qquad \delta_{\text{sample}} = \delta_E + \varepsilon_\delta$$

$$\varepsilon_\phi, \varepsilon_\delta \ \sim\ U\left(-\frac{W\pi}{360}, \frac{W\pi}{360}\right)$$

The two offsets are independent. Boxes crossing either pole are rejected. Conversion fixes the center at the trigger time once; background timestamps do not rotate it. antenna_patterns() evaluates exactly the supplied center, not an average over this box.

What remains separate

Arrival delays require detector positions and are not calculated by antenna_patterns(). A fixed center fixes the geometric response during a short signal; the waveform polarizations can still vary in time.

5. Polarization and detector tensors

Code: u, v, m, n and DETECTOR_ARMS

Construct two orthonormal vectors perpendicular to the source direction. u points toward increasing polar angle (south); v points toward increasing azimuth (east).

$$\hat{u} = (\cos\phi\sin\delta_E,\ \sin\phi\sin\delta_E,\ -\cos\delta_E)$$

$$\hat{v} = (-\sin\phi,\ \cos\phi,\ 0)$$

The local ml4gw/LAL polarization convention chooses the following basis at angle psi. The code reduces psi modulo pi because the tensor response repeats after a half-turn.

$$\hat{m} = -\hat{u}\sin\psi - \hat{v}\cos\psi, \qquad \hat{n} = -\hat{u}\cos\psi + \hat{v}\sin\psi$$

$$e_{ij}^{+} = m_i m_j - n_i n_j, \qquad e_{ij}^{\times} = m_i n_j + n_i m_j$$

The two perpendicular detector arms

For each detector, x and y are the unit vectors along its two arms, expressed in the same Earth-fixed Cartesian frame as s. Their differential response is the detector tensor D:

$$D_{ij} = \frac{1}{2}(x_i x_j - y_i y_j)$$

The code uses these fixed LAL arm-vector constants (also used by local ml4gw):

```
H1 x = (-0.22389266154, +0.79983062746, +0.55690487831)
H1 y = (-0.91397818574, +0.02609403989, -0.40492342125)

L1 x = (-0.95457412153, -0.14158077340, -0.26218911324)
L1 y = (+0.29774156894, -0.48791033647, -0.82054461286)
```

The same source basis is projected onto different arm vectors at Hanford and Livingston. This is why each detector gets a different pair of coefficients, even before accounting for arrival delay or detector noise.

6. Antenna coefficients

Code: dot(), f_plus, f_cross, response_amplitude

Contract the two tensors

$$F_+ = \sum_{i,j} D_{ij} e^+_{ij}, \qquad F_\times = \sum_{i,j} D_{ij} e^\times_{ij}$$

The implementation uses four dot products rather than constructing full matrices. Define a = x dot m, b = x dot n, c = y dot m, and d_arm = y dot n.

$$F_+ = \tfrac{1}{2}(a^2 - b^2 - c^2 + d^2_{\text{arm}}), \qquad F_\times = ab - cd_{\text{arm}}$$

$$R = \sqrt{F_+^2 + F_\times^2}$$

The returned keys are F_plus, F_cross and response_amplitude (= R), separately for H1 and L1. These dimensionless long-wavelength geometric factors are not efficiency, SNR, strain amplitude or a noise-weighted sensitivity.

How the coefficients multiply a waveform

$$h_I(t) = F_{+,I}\, h_+(t - \tau_I) + F_{\times,I}\, h_\times(t - \tau_I)$$

Here I denotes H1 or L1; tau_I is the arrival delay relative to a chosen reference time. The current function returns only the F coefficients and R. A neutrino event supplies a hypothetical GW direction/time, not evidence that a GW was observed.

Why R does not depend on the polarization angle

$$F_+(\psi) = F_+(0)\cos 2\psi + F_\times(0)\sin 2\psi$$

$$F_\times(\psi) = F_\times(0)\cos 2\psi - F_+(0)\sin 2\psi$$

These equations are a rotation of the two-component response vector, so its squared length is unchanged. Individual coefficients can change sign or magnitude as psi changes, while R stays fixed. R is a geometric summary, not the response to every possible polarization waveform.

The full-sky plots vary phi and declination to show the pattern. The star and numerical callout give the response at the event itself; those numbers are evaluated directly, not read back from the colored grid.

7. Worked example and verification

IceCube IC110514A | reference polarization psi = 0

```
RA = 138.47 deg, Dec = -1.94 deg
UTC = 2011-05-14T01:32:22.654109
DUT1 = -0.26010784327993 s; TAI-UTC = 34 s
```

Step	Calculated value
Julian date at midnight	2455695.5
UT1 fraction of day	0.06414807871709
TT centuries T	0.11363627425346
Earth rotation angle	4.440134259756 rad
GMST	4.442675365872 rad
Earth phi	243.9234517888 deg
Earth declination / polar angle	-1.94 deg / 91.94 deg

Detector	F_plus	F_cross	R
H1	-0.26037035	+0.62222547	+0.67450519
L1	+0.49816080	-0.58383091	+0.76747815

What was checked

Both coordinate functions agree for 348 IceCube catalog rows, two GRBs and five date-boundary cases: maximum angle difference 3.553e-14 rad. The comparison shares measured DUT1, but no reference sidereal time, Julian date or output angle is fed to the arithmetic calculation.

Antenna coefficients agree with local ml4gw and LALSuite across 262 sky/polarization cases to absolute tolerance 2e-7 (reference tensors use float32). Tests cover signs, poles, polarization rotation, compatibility and output plots. These checks validate implementation consistency within the stated coordinate convention.

8. References and usage

How to reproduce the calculation and relate the document to the code

```
from gwak.sky_position_antenna_patterns import (
    event_to_gwak, event_to_gwak_from_first_principles,
    antenna_patterns, plot_antenna_patterns)

a = event_to_gwak(138.47, -1.94,
                  "2011-05-14T01:32:22.654109")
b = event_to_gwak_from_first_principles(
    138.47, -1.94, "2011-05-14T01:32:22.654109",
    ut1_minus_utc_seconds=a["ut1_minus_utc_seconds"])
response = antenna_patterns(b, polarization_rad=0.0)
plot_antenna_patterns(b, name="IC110514A",
                      output="IC110514A_antenna")
```

The example deliberately shares only the measured DUT1 scalar. The independent conversion receives no converted angles or time values from the Astropy path. antenna_patterns accepts either a or b unchanged.

Primary formula references

IAU 2006 GMST and Earth rotation angle, ERFA implementation:
https://raw.githubusercontent.com/liberfa/erfa/master/src/gmst06.c
https://raw.githubusercontent.com/liberfa/erfa/master/src/era00.c

IERS leap-second table:
https://hpiers.obspm.fr/iers/bul/bulc/Leap_Second.dat

LAL detector response and arm vectors:
https://lscsoft.docs.ligo.org/lalsuite/7.26/lal/_det_response_8c_source.html
https://lscsoft.docs.ligo.org/lalsuite/7.26/lal/_l_a_l_detectors_8h_source.html